JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Year

java.lang.Object
    java.time.Year

All Implemented Interfaces:

`Serializable`, `Comparable<Year>`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class Year
extends Object
implements Temporal, TemporalAdjuster, Comparable<Year>, Serializable
```

A year in the ISO-8601 calendar system, such as `2007`.

`Year` is an immutable date-time object that represents a year. Any field that can be derived from a year can be obtained.

Note that years in the ISO chronology only align with years in the Gregorian-Julian system for modern years. Parts of Russia did not switch to the modern Gregorian/ISO rules until 1920. As such, historical years must be treated with caution.

This class does not store or represent a month, day, time or time-zone. For example, the value "2007" can be stored in a `Year`.

Years represented by this class follow the ISO-8601 standard and use the proleptic numbering system. Year 1 is preceded by year 0, then by year -1.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time. For most applications written today, the ISO-8601 rules are entirely suitable. However, any application that makes use of historical dates, and requires them to be accurate will find the ISO-8601 approach unsuitable.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	`MAX_VALUE`	The maximum supported year, '+999,999,999'.
`static final int`	`MIN_VALUE`	The minimum supported year, '-999,999,999'.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`Temporal`	`adjustInto(Temporal temporal)`	Adjusts the specified temporal object to have this year.
`LocalDate`	`atDay(int dayOfYear)`	Combines this year with a day-of-year to create a `LocalDate`.
`YearMonth`	`atMonth(int month)`	Combines this year with a month to create a `YearMonth`.
`YearMonth`	`atMonth(Month month)`	Combines this year with a month to create a `YearMonth`.
`LocalDate`	`atMonthDay(MonthDay monthDay)`	Combines this year with a month-day to create a `LocalDate`.
`int`	`compareTo(Year other)`	Compares this year to another year.
`boolean`	`equals(Object obj)`	Checks if this year is equal to another year.
`String`		

	format(DateTimeFormatter forma	Formats this year using the specified formatter.
static Year	**from**(TemporalAccessor tempor	Obtains an instance of Year from a temporal object.
int	**get**(TemporalField field)	Gets the value of the specified field from this year as an int.
long	**getLong**(TemporalField field)	Gets the value of the specified field from this year as a long.
int	**getValue**()	Gets the year value.
int	**hashCode**()	A hash code for this year.
boolean	**isAfter**(Year other)	Checks if this year is after the specified year.
boolean	**isBefore**(Year other)	Checks if this year is before the specified year.
boolean	**isLeap**()	Checks if the year is a leap year, according to the ISO proleptic calendar system rules.
static boolean	**isLeap**(long year)	Checks if the year is a leap year, according to the ISO proleptic calendar system rules.
boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
boolean	**isSupported**(TemporalUnit unit)	Checks if the specified unit is supported.
boolean	**isValidMonthDay**(MonthDay monthDay)	Checks if the month-day is valid for this year.
int	**length**()	Gets the length of this year in days.

Year	**minus**(long amountToSubtract, TemporalUnit unit)	Returns a copy of this year with the specified amount subtracted.
Year	**minus**(TemporalAmount amountTo	Returns a copy of this year with the specified amount subtracted.
Year	**minusYears**(long yearsToSubtract)	Returns a copy of this Year with the specified number of years subtracted.
static Year	**now**()	Obtains the current year from the system clock in the default time-zone.
static Year	**now**(Clock clock)	Obtains the current year from the specified clock.
static Year	**now**(ZoneId zone)	Obtains the current year from the system clock in the specified time-zone.
static Year	**of**(int isoYear)	Obtains an instance of Year.
static Year	**parse**(CharSequence text)	Obtains an instance of Year from a text string such as 2007.
static Year	**parse**(CharSequence text, DateTimeFormatter format	Obtains an instance of Year from a text string using a specific formatter.
Year	**plus**(long amountToAdd, TemporalUnit unit)	Returns a copy of this year with the specified amount added.
Year	**plus**(TemporalAmount amountTo	Returns a copy of this year with the specified amount added.
Year	**plusYears**(long yearsToAdd)	Returns a copy of this Year with the specified number of years added.

<R> R	**query**(TemporalQuery<R> query)	Queries this year using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
String	**toString**()	Outputs this year as a String.
long	**until**(Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another year in terms of the specified unit.
Year	**with**(TemporalAdjuster adjust	Returns an adjusted copy of this year.
Year	**with**(TemporalField field, long newValue)	Returns a copy of this year with the specified field set to a new value.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

MIN_VALUE

```
public static final int MIN_VALUE
```

The minimum supported year, '-999,999,999'.

See Also:

Constant Field Values

MAX_VALUE

```
public static final int MAX_VALUE
```

The maximum supported year, '+999,999,999'.

See Also:

Constant Field Values

Method Details

now

`public static Year now()`

Obtains the current year from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current year.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current year using the system clock and default time-zone, not null

now

`public static Year now(ZoneId zone)`

Obtains the current year from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current year. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current year using the system clock, not null

now

`public static Year now(Clock clock)`

Obtains the current year from the specified clock.

This will query the specified clock to obtain the current year. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current year, not null

of

`public static Year of(int isoYear)`

Obtains an instance of `Year`.

This method accepts a year value from the proleptic ISO calendar system.

The year 2AD/CE is represented by 2.
The year 1AD/CE is represented by 1.
The year 1BC/BCE is represented by 0.
The year 2BC/BCE is represented by -1.

Parameters:

`isoYear` - the ISO proleptic year to represent, from `MIN_VALUE` to `MAX_VALUE`

Returns:

the year, not null

Throws:

`DateTimeException` - if the field is invalid

from

`public static Year from(TemporalAccessor temporal)`

Obtains an instance of `Year` from a temporal object.

This obtains a year based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `Year`.

The conversion extracts the `year` field. The extraction is only permitted if the temporal object has an ISO chronology, or can be converted to a `LocalDate`.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `Year::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the year, not null

Throws:

`DateTimeException` - if unable to convert to a `Year`

parse

`public static Year parse(CharSequence text)`

Obtains an instance of `Year` from a text string such as `2007`.

The string must represent a valid year.

Parameters:

`text` - the text to parse such as "2007", not null

Returns:

the parsed year, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static Year parse(CharSequence text,
                         DateTimeFormatter formatter)
```

Obtains an instance of `Year` from a text string using a specific formatter.

The text is parsed using the formatter, returning a year.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed year, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isLeap

```
public static boolean isLeap(long year)
```

Checks if the year is a leap year, according to the ISO proleptic calendar system rules.

This method applies the current rules for leap years across the whole time-line. In general, a year is a leap year if it is divisible by four without remainder. However, years divisible by 100, are not leap years, with the exception of years divisible by 400 which are.

For example, 1904 is a leap year it is divisible by 4. 1900 was not a leap year as
it is divisible by 100, however 2000 was a leap year as it is divisible by 400.

The calculation is proleptic - applying the same rules into the far future and far past. This is historically inaccurate, but is correct for the ISO-8601 standard.

Parameters:

`year` - the year to check

Returns:

true if the year is leap, false otherwise

getValue

```
public int getValue()
```

Gets the year value.

The year returned by this method is proleptic as per `get(YEAR)`.

Returns:

the year, `MIN_VALUE` to `MAX_VALUE`

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this year can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `YEAR_OF_ERA`
- `YEAR`
- `ERA`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this year, false if not

isSupported

```
public boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this year. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `YEARS`
- `DECADES`
- `CENTURIES`
- `MILLENNIA`
- `ERAS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit` - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This year is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this year as an `int`.

This queries this year for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this year. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this year as a `long`.

This queries this year for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this year. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

isLeap

```
public boolean isLeap()
```

Checks if the year is a leap year, according to the ISO proleptic calendar system rules.

This method applies the current rules for leap years across the whole time-line. In general, a year is a leap year if it is divisible by four without remainder. However, years divisible by 100, are not leap years, with the exception of years divisible by 400 which are.

For example, 1904 is a leap year it is divisible by 4. 1900 was not a leap year as
it is divisible by 100, however 2000 was a leap year as it is divisible by 400.

The calculation is proleptic - applying the same rules into the far future and far past. This is historically inaccurate, but is correct for the ISO-8601 standard.

Returns:

true if the year is leap, false otherwise

isValidMonthDay

```
public boolean isValidMonthDay(MonthDay monthDay)
```

Checks if the month-day is valid for this year.

This method checks whether this year and the input month and day form a valid date.

Parameters:

`monthDay` - the month-day to validate, null returns false

Returns:

true if the month and day are valid for this year

length

```
public int length()
```

Gets the length of this year in days.

Returns:

the length of this year in days, 365 or 366

with

```
public Year with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this year.

This returns a `Year`, based on this one, with the year adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a `Year` based on `this` with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public Year with(TemporalField field,
                 long newValue)
```

Returns a copy of this year with the specified field set to a new value.

This returns a `Year`, based on this one, with the value for the specified field changed. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the adjustment is implemented here. The supported fields behave as follows:

- `YEAR_OF_ERA` - Returns a `Year` with the specified year-of-era The era will be unchanged.
- `YEAR` - Returns a `Year` with the specified year. This completely replaces the date and is equivalent to `of(int)`.
- `ERA` - Returns a `Year` with the specified era. The year-of-era will be unchanged.

In all cases, if the new value is outside the valid range of values for the field then a `DateTimeException` will be thrown.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the

argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

a `Year` based on `this` with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

plus

```
public Year plus(TemporalAmount amountToAdd)
```

Returns a copy of this year with the specified amount added.

This returns a `Year`, based on this one, with the specified amount added. The amount is typically `Period` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

a `Year` based on this year with the addition made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public Year plus(long amountToAdd,
                 TemporalUnit unit)
```

Returns a copy of this year with the specified amount added.

This returns a `Year`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented here. The supported fields behave as follows:

- `YEARS` - Returns a `Year` with the specified number of years added. This is equivalent to `plusYears(long)`.
- `DECADES` - Returns a `Year` with the specified number of decades added. This is equivalent to calling `plusYears(long)` with the amount multiplied by 10.
- `CENTURIES` - Returns a `Year` with the specified number of centuries added. This is equivalent to calling `plusYears(long)` with the amount multiplied by 100.
- `MILLENNIA` - Returns a `Year` with the specified number of millennia added. This is equivalent to calling `plusYears(long)` with the amount multiplied by 1,000.
- `ERAS` - Returns a `Year` with the specified number of eras added. Only two eras are supported so the amount must be one, zero or minus one. If the amount is non-zero then the year is changed such that the year-of-era is unchanged.

All other `ChronoUnit` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

a `Year` based on this year with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusYears

```
public Year plusYears(long yearsToAdd)
```

Returns a copy of this `Year` with the specified number of years added.

This instance is immutable and unaffected by this method call.

Parameters:

`yearsToAdd` - the years to add, may be negative

Returns:

a `Year` based on this year with the years added, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

minus

```
public Year minus(TemporalAmount amountToSubtract)
```

Returns a copy of this year with the specified amount subtracted.

This returns a `Year`, based on this one, with the specified amount subtracted. The amount is typically `Period` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a `Year` based on this year with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public Year minus(long amountToSubtract,
                  TemporalUnit unit)
```

Returns a copy of this year with the specified amount subtracted.

This returns a `Year`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a `Year` based on this year with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusYears

```
public Year minusYears(long yearsToSubtract)
```

Returns a copy of this `Year` with the specified number of years subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`yearsToSubtract` - the years to subtract, may be negative

Returns:

a `Year` based on this year with the year subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this year using the specified query.

This queries this year using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this year.

This returns a temporal object of the same observable type as the input with the year changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` passing `ChronoField.YEAR` as the field. If the specified temporal object does not use the ISO calendar system then a `DateTimeException` is thrown.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisYear.adjustInto(temporal);
temporal = temporal.with(thisYear);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another year in terms of the specified unit.

This calculates the amount of time between two `Year` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified year. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `Year` using `from(TemporalAccessor)`. For example, the amount in decades between two year can be calculated using `startYear.until(endYear, DECADES)`.

The calculation returns a whole number, representing the number of complete units between the two years. For example, the amount in decades between 2012 and 2031 will only be one decade as it is one year short of two decades.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, YEARS);
amount = YEARS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive` - the end date, exclusive, which is converted to a `Year`, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this year and the end year

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to a `Year`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

format

```
public String format(DateTimeFormatter formatter)
```

Formats this year using the specified formatter.

This year will be passed to the formatter to produce a string.

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted year string, not null

Throws:

DateTimeException - if an error occurs during printing

atDay

`public LocalDate atDay(int dayOfYear)`

Combines this year with a day-of-year to create a `LocalDate`.

This returns a `LocalDate` formed from this year and the specified day-of-year.

The day-of-year value 366 is only valid in a leap year.

Parameters:

`dayOfYear` - the day-of-year to use, from 1 to 365-366

Returns:

the local date formed from this year and the specified date of year, not null

Throws:

DateTimeException - if the day of year is zero or less, 366 or greater or equal to 366 and this is not a leap year

atMonth

`public YearMonth atMonth(Month month)`

Combines this year with a month to create a `YearMonth`.

This returns a `YearMonth` formed from this year and the specified month. All possible combinations of year and month are valid.

This method can be used as part of a chain to produce a date:

```
LocalDate date = year.atMonth(month).atDay(day);
```

Parameters:

`month` - the month-of-year to use, not null

Returns:

the year-month formed from this year and the specified month, not null

atMonth

`public YearMonth atMonth(int month)`

Combines this year with a month to create a `YearMonth`.

This returns a `YearMonth` formed from this year and the specified month. All possible combinations of year and month are valid.

This method can be used as part of a chain to produce a date:

```
LocalDate date = year.atMonth(month).atDay(day);
```

Parameters:

`month` - the month-of-year to use, from 1 (January) to 12 (December)

Returns:

the year-month formed from this year and the specified month, not null

Throws:

`DateTimeException` - if the month is invalid

atMonthDay

`public LocalDate atMonthDay(MonthDay monthDay)`

Combines this year with a month-day to create a `LocalDate`.

This returns a `LocalDate` formed from this year and the specified month-day.

A month-day of February 29th will be adjusted to February 28th in the resulting date if the year is not a leap year.

Parameters:

`monthDay` - the month-day to use, not null

Returns:

the local date formed from this year and the specified month-day, not null

compareTo

`public int compareTo(Year other)`

Compares this year to another year.

The comparison is based on the value of the year. It is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable<Year>`

Parameters:

`other` - the other year to compare to, not null

Returns:

the comparator value, that is less than zero if this is before `other`, zero if they are equal, or greater than zero if this is after `other`

See Also:

`isBefore(Year)`, `isAfter(Year)`

isAfter

```
public boolean isAfter(Year other)
```

Checks if this year is after the specified year.

Parameters:

`other` - the other year to compare to, not null

Returns:

true if this is after the specified year

isBefore

```
public boolean isBefore(Year other)
```

Checks if this year is before the specified year.

Parameters:

`other` - the other year to compare to, not null

Returns:

true if this point is before the specified year

equals

```
public boolean equals(Object obj)
```

Checks if this year is equal to another year.

The comparison is based on the time-line position of the years.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other year

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this year.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this year as a `String`.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this year, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.